เปิดฝากระโปรง LLM: โครงสร้าง การเทรน และการ Prompt

LLM & Agentic AI · สัปดาห์ 3 · ช่วงเช้า (ทฤษฎี)

ดร.ธีรพงศ์ อรชร

วันนี้เรียนอะไร

1 ช่วงเช้า · ทฤษฎี

ส่วนที่ 1 · ผ่าโครงสร้าง LLM (Typhoon2-1B) + สาธิตสด Lab 1

ส่วนที่ 2 · การเทรน การ fine-tune และการเตรียมข้อมูล

ส่วนที่ 3 · Prompt ทำงานอย่างไรในโมเดล — prompt ดี vs ไม่ดี ทีละขั้น

2 ช่วงบ่าย · ปฏิบัติ (Colab)

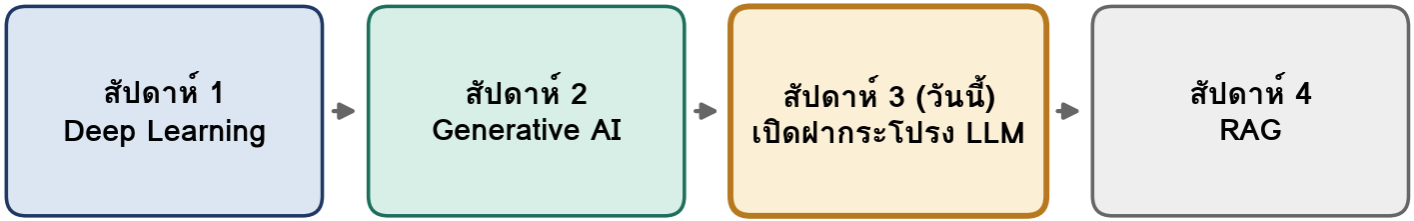
Lab 2 · ตามรอย prompt 3 ระดับ (คลุมเครือ / สั้น / เต็ม) วัดด้วยตัวชี้วัดเดียวกัน

Lab 3 · Prompt engineering อย่างเป็นระบบ + เทียบ ChatGPT/Claude

Lab 4 · เตรียมข้อมูลและ fine-tune ด้วย LoRA

เชื่อมโยง

จากสองสัปดาห์ก่อน สู่วันนี้



- สัปดาห์ 1: neural network เรียนรู้อย่างไร
- สัปดาห์ 2: Transformer และ LLM ในระดับแนวคิด
- วันนี้: หยิบโมเดลจริงมาผ่าให้เห็นว่าทั้งหมดประกอบกันอย่างไร

เส้นด้ายของวัน

ประโยคเดียวที่จะใช้ทั้งวัน

LLM คือฟังก์ชันที่รับ token เข้าไป
แล้วทำนายความน่าจะเป็นของ token ถัดไป

การ fine-tune = เปลี่ยนตัวฟังก์ชัน (weight)
การ prompt = เปลี่ยนสิ่งที่ป้อนเข้าฟังก์ชัน (input)
ทุกหัวข้อวันนี้จะกลับมาที่ประโยคนี้

ส่วนที่ 1

ผ่าโครงสร้าง LLM

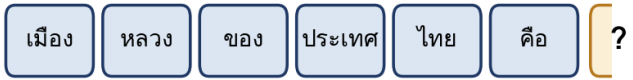
Typhoon2-1B ประกอบจากอะไร · อ่าน config อย่างไร · นับพารามิเตอร์ด้วยมือ · Typhoon2 เกิดจาก Llama อย่างไร

1

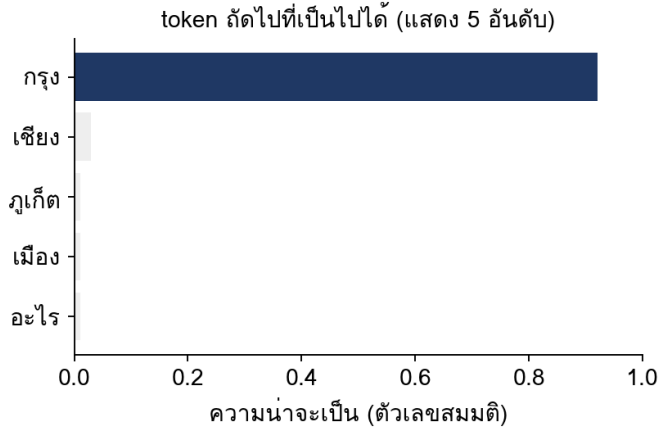
ขั้นที่ 1

ทบทวน: LLM ทำอะไรกันแน่

ป้อน token ทั้งหมดที่มี → โมเดลให้ความน่าจะเป็นของ token ถัดไป



เลือกมา 1 token ต่อท้าย แล้วทำซ้ำ = การ "เขียน" ทีละ token (Lab 2.2 สัปดาห์ 2)

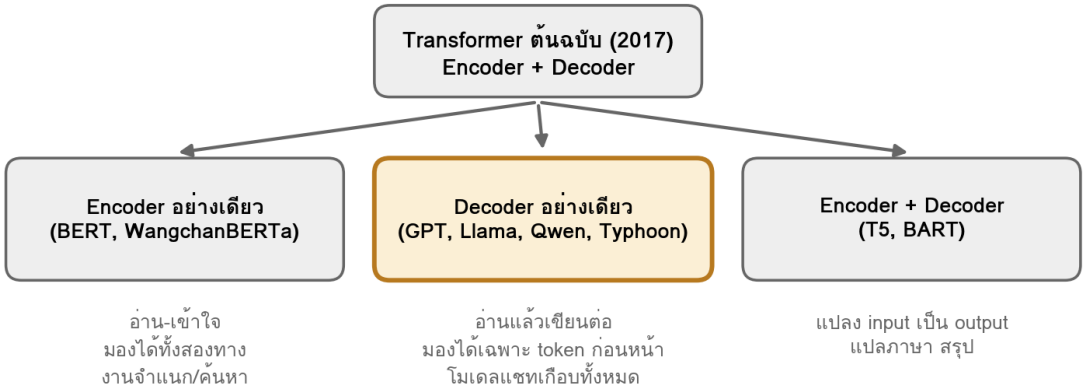


- รับ token ทั้งหมดที่มี → ให้ความน่าจะเป็นของ token ถัดไป → เลือก 1 token ต่อท้าย แล้วทำซ้ำ (Lab 2.2 สัปดาห์ 2)

- โมเดลไม่ได้ "ค้นหาความจริง" มันทำนายรูปแบบที่เคยเห็น — รากของ hallucination และเหตุผลที่ prompt มีผล

ขั้นที่ 2

ทำไมโมเดลแชทเกือบทั้งหมดเป็น decoder-only

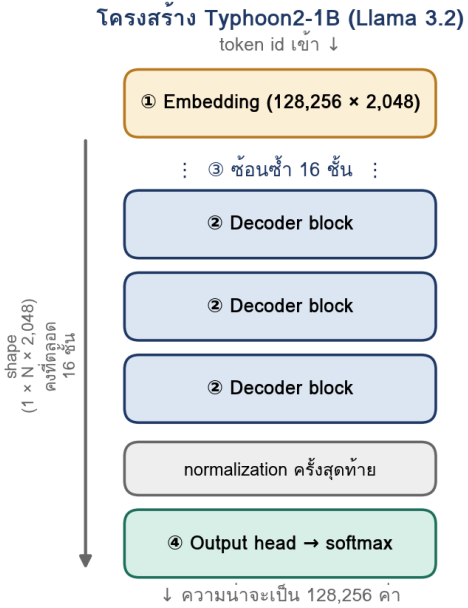


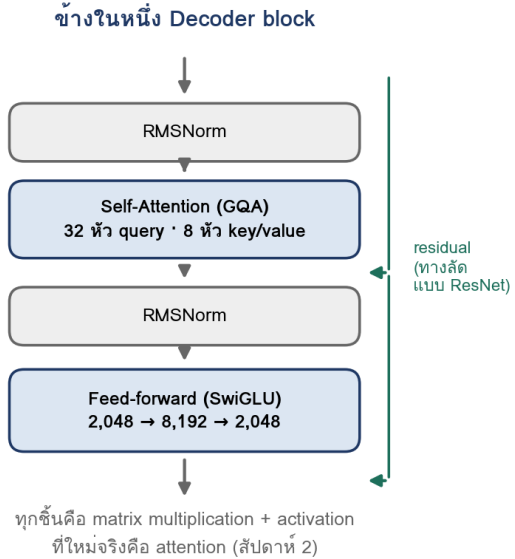
วันนี้เจาะสายกลาง: โมเดลชนิดเดียวใช้ได้ทุกงาน เพียงเปลี่ยนสิ่งที่ป้อนเข้าไป

- งานแชทคือ "อ่านแล้วเขียนต่อ" ซึ่ง decoder ทำได้คนเดียวจบ: อ่านทุกอย่างที่มีผ่าน attention แล้วเขียนต่อทีละ token
- โมเดลชนิดเดียวใช้ได้ทุกงาน เพียงเปลี่ยนสิ่งที่ป้อนเข้าไป

ขั้นที่ 3

ประกอบโมเดลทีละชิ้น: มีแค่ 4 ประเภท





- ① Embedding → ② Decoder block (Attention + Feed-forward) → ③ ซ้อน 16 ชั้น + normalization ครั้งสุดท้าย → ④ Output head + softmax

ขั้นที่ 3

แต่ละชิ้นคืออะไร (ภาษาคน)

1 ① Embedding

ตารางแปลง token id → vector ยาว 2,048
ขนาด 128,256 × 2,048

เป็นชั้นแรกสุด คล้าย vector ของคำที่วาดเทียบใน Lab 1.3 สัปดาห์ 2

2 ② Decoder block

Attention: แต่ละ token "มองหา" token อื่นที่เกี่ยวข้อง (สัปดาห์ 2)

Feed-forward: neural network ธรรมดา ประมวลผลแต่ละ token แยกกัน

คั่นด้วย normalization + residual

3 ③ ซ้อน 16 ชั้น

block เดิมซ้ำ 16 ครั้ง = "deep"

เหมือน CNN ซ้อน conv layer หลายชั้น (สัปดาห์ 1)

ปิดด้วย normalization ครั้งสุดท้าย

4 ④ Output head

แปลง vector กลับเป็นคะแนนของทุก token ใน vocab แล้ว softmax

softmax ตัวเดียวกับที่จำแนก 10 คลาสใน Fashion-MNIST — แต่ตอนนี้ 128,256 คลาส

ขั้นที่ 3

Attention คืออะไร — เหมือนหันไปดูว่าใครพูดอะไรไว้ก่อนหน้า

สถานการณ์

ในห้องประชุมมีคนพูดว่า "มันแพง"

คุณสงสัยว่า "มัน" คืออะไร สมองจึงหันกลับไปดูว่าก่อนหน้านี้ใครพูดถึงอะไรไว้ แล้วหยิบคำที่น่าจะใช่มาเติมความหมาย

ในโมเดล

token "มัน" ทำแบบเดียวกัน: มองย้อนไปทุก token ก่อนหน้า ให้คะแนนว่าใครเกี่ยวข้อง แล้วดึงความหมายของตัวที่คะแนนสูงมาผสมกับตัวเอง

คะแนนที่รวมกันได้ 1 นี่แหละคือ "attention" ที่ Lab 2 จะวาดกราฟ

ทำไมสำคัญกับ prompt

token ดึงได้เฉพาะสิ่งที่อยู่ในห้อง (context)

สิ่งที่ไม่ได้เขียนไว้ใน prompt ไม่มีให้ดึง โมเดลต้องเดาจากความจำตอนเทรน → hallucination

ขั้นที่ 3

Q · K · V · O: สี่ตารางใน attention ทำหน้าที่อะไร

Q = คำถาม

"ฉันกำลังหาอะไร"

token "มัน" ถามว่า มีของอะไรถูกพูดถึงก่อนหน้านี้บ้าง

K = ป้ายชื่อ

"ฉันมีอะไรให้ค้นเจอ"

ทุก token ชูป้าย: "กาแฟ" = สิ่งของ, "ดี" = คำบรรยาย · เอาคำถามไปเทียบป้าย ป้ายไหนตรงได้คะแนนสูง

V = เนื้อหา

"ถ้าเลือกฉัน จะได้ข้อมูลนี้ไป"

พอเลือก "กาแฟ" สิ่งที่ได้คือความหมายจริงของกาแฟ ไม่ใช่ป้าย (เหมือนสันหนังสือ vs เนื้อใน)

O = สรุปกลับเข้าตัว

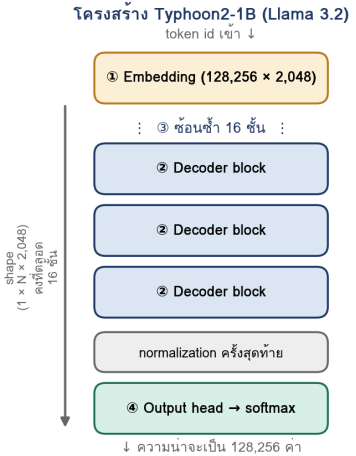
รวมผลจากทุกหัวแล้วเขียนกลับลงใน "มัน"

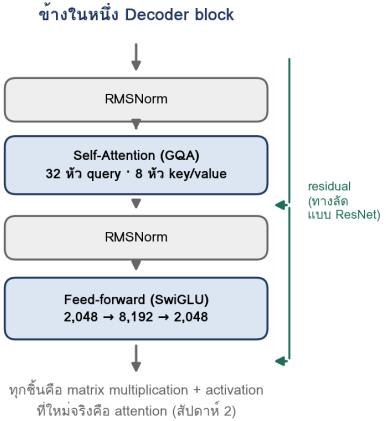
ตอนนี้ "มัน" = มัน (ที่หมายถึงกาแฟ) ส่งต่อไปชั้นถัดไป

ขั้นที่ 3

Feed-forward = neural network แบบสัปดาห์ 1

- **สัปดาห์ 1 (Fashion-MNIST)**
  - Dense(128, relu) → Dense(10): ขยาย 784 → 128 → หด 10
- **ใน Typhoon2 (ทุกชั้น)**
  - ขยาย 2,048 → 8,192 → หดกลับ 2,048 — สองชั้น Dense มี activation คั่น แค่ตัวเลขใหญ่กว่า
- **ต่างกัน 3 จุดเล็ก ๆ**
  - ไม่มี bias · activation เป็น SiLU (ReLU โค้งนุ่ม) · มี "ประตู" จึงเป็น 3 ตารางแทน 2 (SwiGLU)
- **หน้าที่ต่างจาก attention**
  - attention = รวบรวมข้อมูลจากเพื่อน · feed-forward = เอาที่รวบรวมมาคิดต่อคนเดียว (ทีละ token ไม่เห็นตัวอื่น)





ขั้นที่ 4

config.json คืออะไร — พิมพ์เขียว ไม่ใช่ตัวบ้าน

ดาวน์โหลดโมเดลได้ 2 อย่าง

ไฟล์ weight: ตัวเลขพันกว่าล้านค่า ≈ 2.5 GB

config.json: ข้อความไม่กี่สิบบรรทัด บอกว่าต้องประกอบโครง network หน้าตาแบบไหน

ไลบรารีทำอะไรตอนโหลด

อ่าน config → สร้างโครงเปล่า → เทตัวเลขจาก weight ลงไป

ถ้าโครงไม่ตรง weight ก็ใส่ไม่ลง — พิมพ์เขียวบ้านกับวัสดุก่อสร้าง

ทำไมต้องอ่านให้เป็น

ทุกค่าใน config คือ "การตัดสินใจออกแบบ" ที่คนเลือกก่อนเทรน

อ่านแล้วรู้ทันทีว่าโมเดลกว้าง/ลึก/รู้จักกี่คำ และคำนวณจำนวนพารามิเตอร์ได้โดยไม่ต้องโหลด weight เลย

ขั้นที่ 4

อ่าน config.json (1/2): ขนาดของตารางต่าง ๆ

ค่าใน config	Typhoon2-1B	ภาษาคน	กำหนดชิ้นไหนในภาพ
architectures	LlamaForCausalLM	ประกอบตามแบบ Llama · CausalLM = ทำนายคำถัดไป มองได้เฉพาะคำก่อนหน้า	ลำดับชิ้นทั้งภาพ
vocab_size	128,256	รู้จัก token กี่แบบ	จำนวนแถวของตาราง Embedding และจำนวนช่องคะแนนของ Output head
hidden_size	2,048	vector ของแต่ละ token ยาวเท่าไร (ความกว้าง) — ไหลผ่านทุกชั้นขนาดไม่เปลี่ยน	ความกว้างของทุกชิ้น
num_hidden_layers	16	ซ้อน block กี่ชั้น (ความลึก)	จำนวนชั้นในกองซ้อน
intermediate_size	8,192	feed-forward ขยาย 2,048 → 8,192 → หดกลับ	ส่วนที่ป่องของ feed-forward
tie_word_embeddings	true	output head ใช้ตาราง embedding ตัวเดิมกลับด้าน ไม่มีตารางที่สอง	เส้นเชื่อม Embedding ↔ Output head

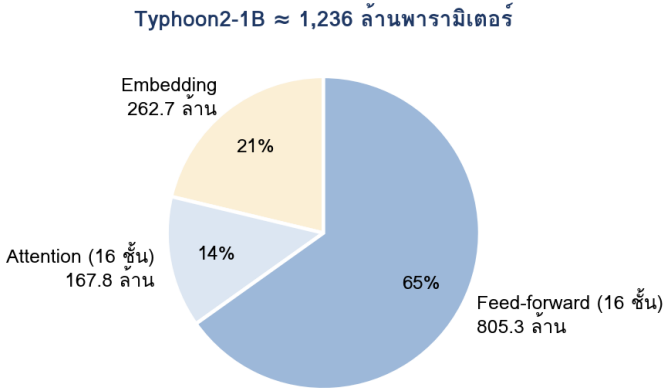
ขั้นที่ 4

อ่าน config.json (2/2): attention และ context

ค่าใน config	Typhoon2-1B	ภาษาคน	กำหนดชิ้นไหนในภาพ
num_attention_heads	32	attention ถาม 32 คำถามพร้อมกัน (แต่ละหัวจับความสัมพันธ์คนละแบบ)	จำนวนช่อง Q
head_dim	64	แต่ละหัวได้ vector ยาวเท่าไร = 2,048 ÷ 32 (Llama ไม่เขียนใน config เพราะคำนวณได้)	ขนาดของช่องเล็ก ๆ ใน Q, K, V
num_key_value_heads	8	ป้ายชื่อ (K) และเนื้อหา (V) มีแค่ 8 ชุด ให้ 32 คำถามแบ่งกันใช้ = GQA	ช่อง K, V (กว้าง 8 × 64 = 512)
max_position_embeddings	131,072	รับ prompt + คำตอบ รวมกันได้ยาวสุดกี่ token	ไม่บรรทัดตำแหน่ง (RoPE)

ขั้นที่ 5

คำนวณจำนวนพารามิเตอร์ด้วยมือ — ทำเหมือนสัปดาห์ 1



วิธีคำนวณด้วยมือ (ทำเหมือน (784 × 128) + 128 ในสัปดาห์ 1)

Embedding: 128,256 × 2,048 ≈ 262.7 ล้าน

Attention ต่อชั้น: Q 2,048×2,048 + K 2,048×512
+ V 2,048×512 + O 2,048×2,048 ≈ 10.5 ล้าน

Feed-forward ต่อชั้น: 3 × 2,048 × 8,192 ≈ 50.3 ล้าน

รวมต่อชั้น ≈ 60.8 ล้าน × 16 ชั้น ≈ 973 ล้าน

รวม ≈ 262.7 + 973 ≈ 1,236 ล้าน ≈ 1.24B ✓

• Llama ไม่ใช้ bias จึงไม่มี "+128" • output head ใช้ตาราง embedding ร่วม

- feed-forward กินพารามิเตอร์มากกว่า attention เกือบ 5 เท่า
- embedding อย่างเดียวคิดเป็นราว 21% — ขนาด vocab จึงเป็นการตัดสินใจที่มีราคา

ขั้นที่ 5

นับพารามิเตอร์ด้วยมือใน 5 ขั้น — กฎเดียว: ตารางกว้าง × ยาว

ขั้น	ชิ้นส่วน	คิดอย่างไร	ผล
1	Embedding	128,256 แถว × 2,048 คอลัมน์	262.7 ล้าน
2	Attention (1 ชั้น)	Q 2,048×2,048 + K 2,048×512 + V 2,048×512 + O 2,048×2,048	10.5 ล้าน
3	Feed-forward (1 ชั้น)	3 ตาราง × 2,048 × 8,192	50.3 ล้าน
4	ซ้อน 16 ชั้น	(10.5 + 50.3) × 16	973 ล้าน
5	รวม	262.7 + 973	1,236 ล้าน ≈ 1.24B ✓

● Feed-forward กิน 65% · Embedding 21% · Attention แค่ 14% — คนมักคิดว่า attention คือตัวใหญ่ แต่ไม่ใช่

ขั้นที่ 5

สี่ข้อที่ต่างจากสูตร (784 × 128) + 128 ของสัปดาห์ 1

- **ไม่มี bias**
  - Llama ไม่ใช้ bias ในชั้น linear จึงไม่มี "+128" — เหลือแค่ผลคูณของขนาดตาราง
- **ไม่นับ normalization (เพราะเล็กมาก)**
  - RMSNorm มีพารามิเตอร์แค่ 2,048 ตัวต่อจุด รวมทั้งโมเดลไม่ถึง 0.01%
- **ไม่นับ output head**
  - tie_word_embeddings = true → ใช้ตาราง embedding ตัวเดียวกัน (ประหยัดไป 262.7 ล้าน)
- **K และ V เล็กกว่า Q**
  - เพราะมีแค่ 8 หัว × 64 = 512 (GQA) ไม่ใช่ 2,048

● รวมทั้งโมเดล ≈ 262.7 + 16 × 60.8 ≈ 1,236 ล้าน ≈ 1.24B — ตรงกับชื่อ "1B" และตรงกับที่โค้ดจะนับได้ใน Lab 1

ขั้นที่ 6

4 เทคนิคที่ Llama ปรับจากต้นฉบับ — จำแค่ "ทำอะไร" พอ

RMSNorm

ตัวปรับขนาด vector ก่อนเข้า attention และ feed-forward (33 จุดทั้งโมเดล)

ถ้าไม่ปรับ ตัวเลขจะโตขึ้นเรื่อย ๆ เมื่อผ่าน 16 ชั้นจนเทรนไม่เสถียร — แนวคิดเดียวกับ Batch Norm สัปดาห์ 1 แต่คำนวณน้อยกว่า

RoPE

วิธีบอก "ตำแหน่ง" ของ token — เพราะ attention เทียบ Q กับ K โดยไม่รู้ลำดับ

แทนที่จะบวกตาราง ตำแหน่ง ใช้วิธี "หมุน" vector ตามตำแหน่ง ไม่มีพารามิเตอร์ และขยาย context ได้ยาว (131k)

GQA

ป้ายชื่อ (K) และเนื้อหา (V) มีแค่ 8 ชุด ให้คำถาม (Q) 32 หัวแบ่งกันใช้

คุ้มเพราะตอน generate ต้องเก็บ K, V ของทุก token เก่าไว้ — เหลือ 8 ชุด หน่วยความจำลด 4 เท่า

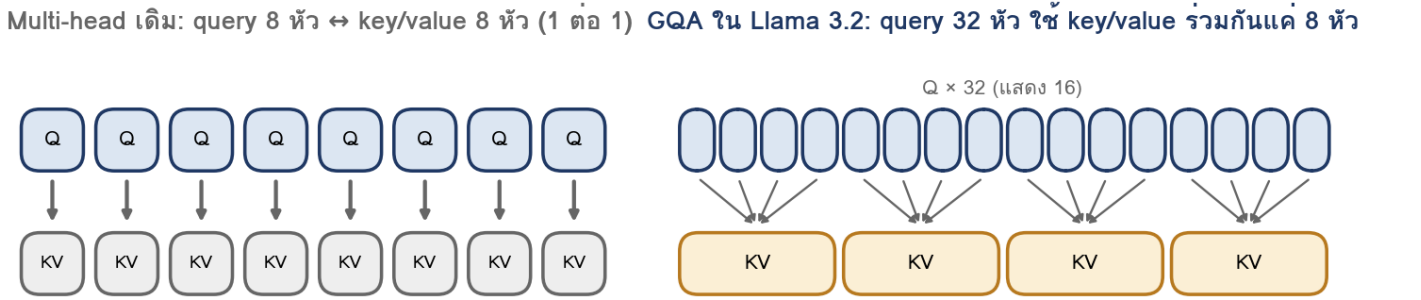
SwiGLU

feed-forward แบบมี "ประตู": ตาราง gate ตัดสินว่าช่องไหนควรเปิด ตาราง up ให้เนื้อหา คูณกันแล้วหดด้วย down

จึงมี 3 ตาราง (50.3 ล้าน/ชั้น) แทน 2 — เรียนรู้ได้ดีกว่าเมื่อพารามิเตอร์เท่ากัน

ขั้นที่ 6

GQA: ทำไมหัว key/value แค่ 8 ก็พอ

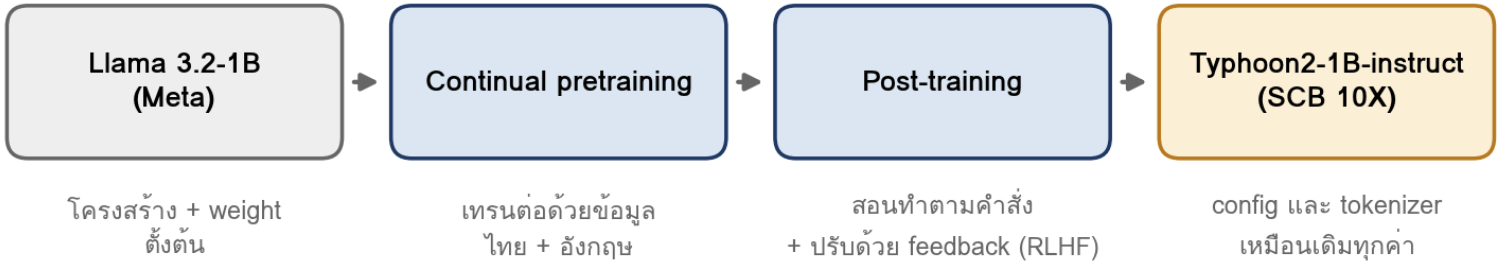


- ตอน generate โมเดลต้องจำ key/value ของทุก token ที่ผ่านมา (KV cache) — token ละ 2 × 16 ชั้น × 8 หัว × 64 × 2 ไบต์ = 32 KB
- context เต็ม 131,072 token ≈ 4.3 GB
- ถ้าไม่มี GQA ≈ 17 GB เกิน GPU T4 (≈ 15 GB)

ขั้นที่ 7

Typhoon2 เกิดจาก Llama อย่างไร

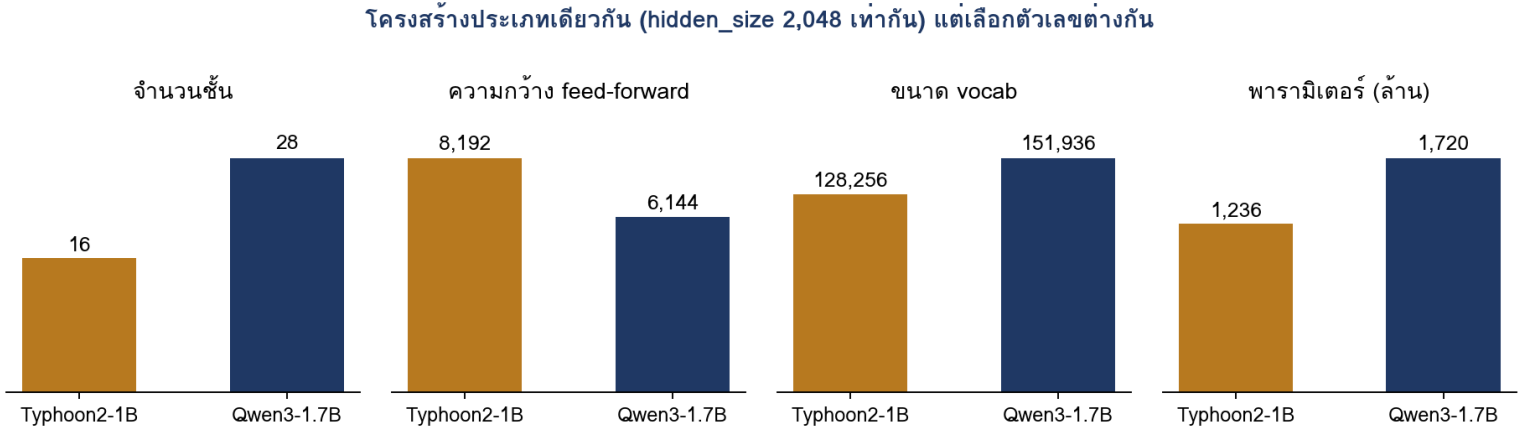
สิ่งที่เปลี่ยนคือ "ค่าใน weight" ไม่ใช่ "โครงสร้าง" — หลักเดียวกับ transfer learning ในสัปดาห์ 1



- Continual pretraining = เทรนต่อด้วยวิธีเดียวกับ pretraining แต่ใช้ข้อมูลไทย+อังกฤษ
- Post-training = สอนทำตามคำสั่ง + ปรับตามความชอบของมนุษย์ (ตระกูลเดียวกับ RLHF สัปดาห์ 2)
- config และ tokenizer เหมือน Llama 3.2-1B ทุกค่า

ขั้นที่ 8

เทียบกับ Qwen3-1.7B: "ดีไซน์" คือการเลือกตัวเลข



- hidden_size เท่ากัน แต่ Qwen3 เลือกชั้นมาก/feed-forward แคบ, หัว attention น้อยแต่ใหญ่ (16 × 128), vocab ใหญ่กว่า

- ขนาดไม่เท่ากัน จึงเทียบได้แค่ "ทางเลือก" ไม่ใช่ "ใครดีกว่า"

ขั้นที่ 8

tokenizer ต่างกัน → ประโยคเดียวกันใช้ token ไม่เท่ากัน



ภาพประกอบเชิงแนวคิด — จำนวนจริงดูใน Lab 1

- ต่อจาก Lab 1.2 สัปดาห์ 2: ภาษาไทยใช้ token มากกว่าภาษาอังกฤษ
- จำนวน token มีผลตรงต่อค่าใช้จ่ายและความยาว context ที่ใช้ได้
- ภาพนี้เป็นเชิงแนวคิด จำนวนจริงดูใน Lab 1

ขั้นที่ 9

แล้ว ChatGPT / Claude ล่ะ — ผ่าดูได้ไหม

สิ่งที่รู้แน่ทั้งสองฝั่ง: เป็น Transformer ที่เทรนให้ทำนาย token ถัดไป

โมเดลเปิด (Typhoon2, Qwen3)	โมเดลปิด (ChatGPT, Claude, Gemini)
เห็น config · เห็น weight · ดู attention ได้ fine-tune เองได้ · รันบนเครื่องเราได้	ไม่รู้จำนวนชั้น · ไม่มี weight · ดู attention ไม่ได้ fine-tune ได้เฉพาะผ่านบริการของผู้ให้บริการ

หลักการวันนี้จึงใช้อธิบายพฤติกรรมของโมเดลปิดได้ — บ่ายนี้จะทดสอบด้วยการยิง prompt เดียวกัน

- โมเดลปิดผ่าไม่ได้ แต่หลักการเดียวกัน (token → ทำนาย token ถัดไป) ใช้อธิบายพฤติกรรมของมันได้ — ช่วงบ่าย Lab 3 จะยิง prompt เดียวกันเข้า Typhoon2 กับ ChatGPT/Claude เทียบกัน

สาธิตสด · Lab 1

ผ่าโมเดลด้วยโค้ด

ผู้สอนรัน notebook บนจอ · นักศึกษาเปิด Lab1_Anatomy_TH.ipynb ตามได้ · ทุกขั้นโยงกลับทฤษฎีที่เพิ่งพูด

1

Lab 1.1 · อ่านพิมพ์เขียวโดยไม่โหลด weight

```
cfg_t = AutoConfig.from_pretrained(TYPHOON)
cfg_q = AutoConfig.from_pretrained(QWEN)

for f in ["vocab_size", "hidden_size",
          "num_hidden_layers", "num_attention_heads",
          "num_key_value_heads", "intermediate_size"]:
    print(f, getattr(cfg_t, f), getattr(cfg_q, f))

#                       Typhoon2 | Qwen3
# vocab_size              128256 | 151936
# hidden_size               2048 |   2048
# num_hidden_layers           16 |     28
# num_attention_heads         32 |     16
# num_key_value_heads          8 |      8
# intermediate_size         8192 |   6144
```

- AutoConfig อ่านแค่ไฟล์ config.json → เร็ว ไม่กินหน่วยความจำ
- เทียบกับตาราง config ในสไลด์ "อ่าน config.json" (ขั้นที่ 4): ตรงทุกค่า
- Qwen3 ต้องใช้ transformers 4.51 ขึ้นไป (มีในเซลล์ติดตั้ง)
  - 1.2 ต่อ: ให้ tokenizer สองตัวตัดประโยคไทยเดียวกัน แล้วนับ token

Lab 1.3–1.4 · โครงสร้างจริง และนับพารามิเตอร์

```
model = AutoModelForCausalLM.from_pretrained(
    TYPHOON, torch_dtype=torch.float16,
    device_map="auto")
print(model)
# embed_tokens: Embedding(128256, 2048)    <- (1)
# layers: ModuleList(0-15)                 <- (2) x16
#   self_attn: q_proj 2048->2048
#              k_proj 2048->512   <- GQA
#   mlp: gate/up 2048->8192, down
# norm: LlamaRMSNorm                 <- (3)
# lm_head: Linear(2048 -> 128256)   <- (4)

# นับพารามิเตอร์แยกตามชิ้น (โค้ดเต็มใน notebook)
# embedding     262.7 ล้าน (21%)
# attention     167.8 ล้าน (14%)
# feed-forward  805.3 ล้าน (65%)
# รวม ≈ 1,236 ล้าน ≈ 1.24B  ✓
```

- โหลดแค่ Typhoon2 ตัวเดียวแบบ float16 (ไม่โหลด Qwen3 เพื่อไม่ให้เกิน RAM ของ Colab ฟรี)
- print(model) เรียงตามชิ้น ①–④ พอดี
- สังเกต k_proj เล็กกว่า q_proj — นั่นคือ GQA
- named_parameters() นับ weight ที่ผูกกัน (tie) ครั้งเดียว

Lab 1.5 · ส่ง tensor ผ่านทีละชิ้น แล้วดู shape

```
text = "เมืองหลวงของประเทศไทยคือ"
enc = tok_t(text, return_tensors="pt")    # (1, N)

# forward hook = ดักดู output ของแต่ละชิ้น
hook_on(model.model.embed_tokens, "(1) embed")
hook_on(model.model.layers[0],    "(2) layer 0")
hook_on(model.model.layers[15],   "(2) layer 15")
hook_on(model.lm_head,            "(4) lm_head")
out = model(**enc)

# (1) embed      (1, N, 2048)
# (2) layer 0    (1, N, 2048)    <- ไม่เปลี่ยน
# (2) layer 15   (1, N, 2048)    <- ตลอด 16 ชั้น
# (4) lm_head    (1, N, 128256) <- ขนาด vocab

probs = softmax(out.logits[0, -1])  # token ถัดไป
```

- shape (1 × N × 2,048) คงที่ตลอด 16 ชั้น — ทุกชั้นรับและส่ง vector ขนาดเดิม
- เปลี่ยนเป็น 128,256 (ขนาด vocab) เฉพาะที่ output head
- logits ของตำแหน่งสุดท้าย เมื่อผ่าน softmax คือ "ความน่าจะเป็นของ token ถัดไป" ในสไลด์ "ประโยคเดียวที่จะใช้ทั้งวัน"
  - 1.6 (เสริม): คำนวณ KV cache ว่า context เต็มใช้กี่ GB

สรุปส่วนที่ 1

หลังจบส่วนที่ 1 นักศึกษาควรพูดได้ว่า

"โมเดลนี้เป็น decoder-only 16 ชั้น กว้าง 2,048 มีพารามิเตอร์ราว 1.2 พันล้าน
ต่อยอดจาก Llama 3.2 ด้วยข้อมูลไทย และทุกชิ้นข้างในคือ neural network
ที่ผมเคยเทรนมาแล้วในสัปดาห์แรก"

ถ้ายังพูดไม่ได้ ให้กลับไปดูสไลด์ 'ประกอบโมเดลทีละชิ้น' และตารางอ่าน config

ส่วนที่ 2

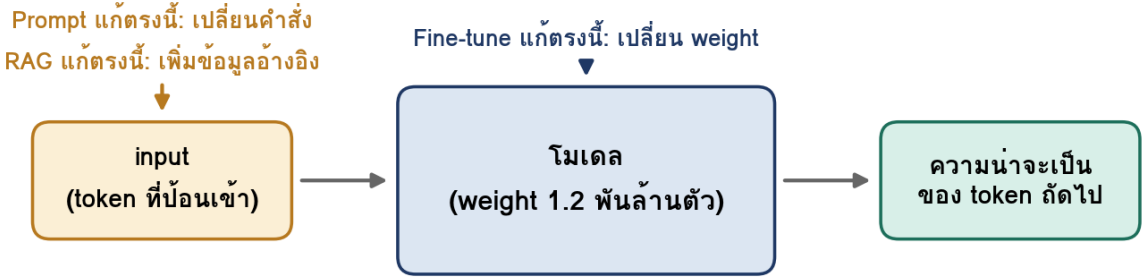
การเทรน การ fine-tune และการเตรียมข้อมูล

งานของเราควร prompt, RAG หรือ fine-tune · LoRA ทำไมเทรนแค่ 0.3% ก็พอ · ข้อมูลต้องหน้าตาอย่างไร

2

ขั้นที่ 1

สามวิธีสอนโมเดล — แก้คนละที่ในสมการเดียวกัน



คำตอบ = โมเดล(input) → มีแค่สองที่ให้แก้: ตัว input หรือ ตัวโมเดล

- คำตอบ = โมเดล(input) จึงมีแค่สองที่ให้แก้: prompt และ RAG แก้ที่ input (คำสั่ง / ข้อมูลอ้างอิง) ส่วน fine-tune แก้ที่ตัวโมเดล (weight)

ขั้นที่ 1

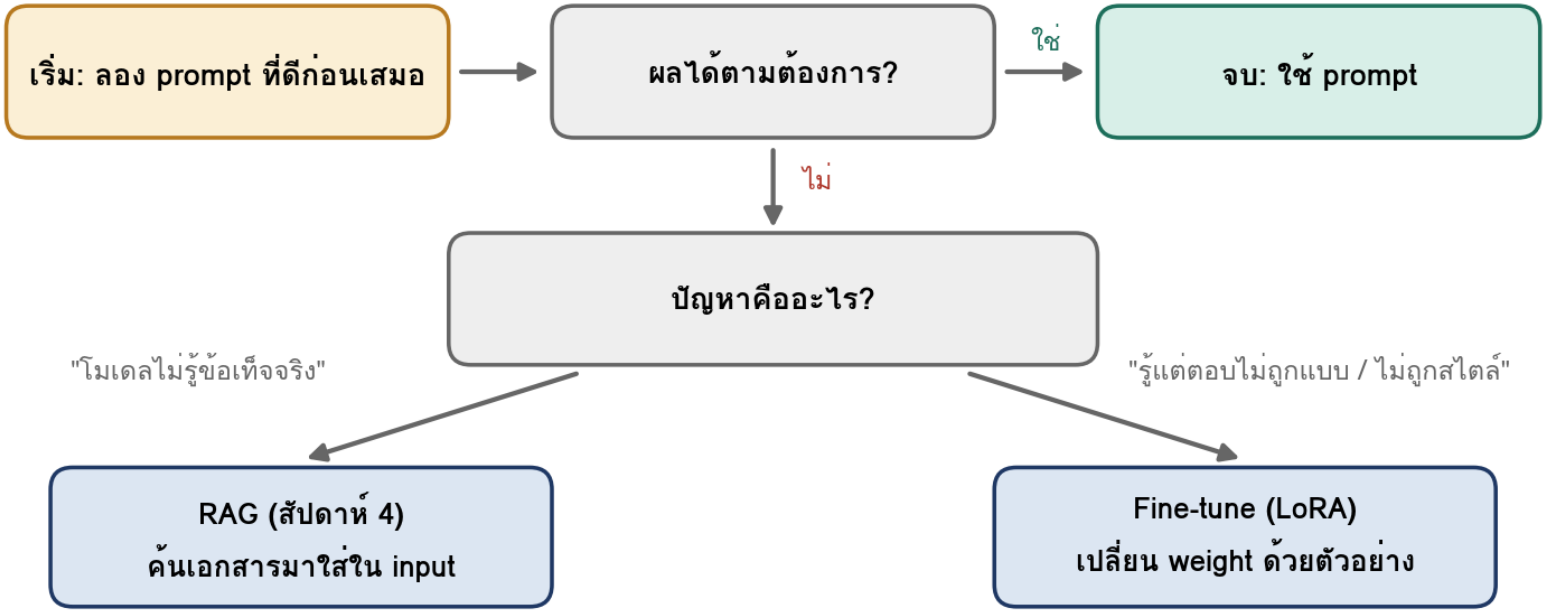
เลือกวิธีให้ตรงปัญหา

วิธี	แก้ที่ไหน	เหมาะกับ	ต้นทุน
Prompt	input — ข้อความคำสั่ง	งานที่โมเดลทำได้อยู่แล้ว แค่ต้องบอกให้ชัด	ต่ำสุด ทำได้ทันที
RAG	input — แนบเอกสารที่ค้นมาได้	ความรู้ที่เปลี่ยนบ่อยหรือมีมาก: คู่มือ กฎระเบียบ ข้อมูลองค์กร	ปานกลาง ต้องมีระบบค้น (สัปดาห์ 4)
Fine-tune	ตัวโมเดล — weight	สไตล์ รูปแบบคำตอบ ทักษะเฉพาะ ที่ prompt แล้วยังไม่ได้	สูงกว่า ต้องเตรียมข้อมูลและเทรน

กฎง่าย ๆ: ลอง prompt ก่อนเสมอ · "โมเดลไม่รู้ข้อเท็จจริง" → ใช้ RAG · "โมเดลรู้แต่ตอบไม่ถูกแบบ" → จึงค่อย fine-tune

ขั้นที่ 1

ลำดับการตัดสินใจ



ขั้นที่ 2

Pretraining vs Fine-tuning: เทรนเหมือนกัน ต่างที่ข้อมูลและจุดเริ่ม

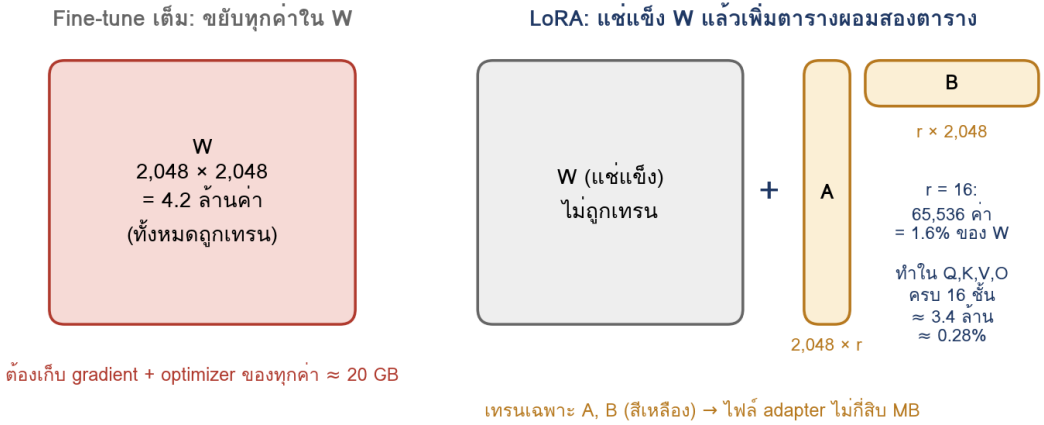
	Pretraining	Fine-tuning (LoRA)
ข้อมูล	ล้านล้าน token (ทั้งอินเทอร์เน็ต)	หลักร้อย–หลักหมื่นตัวอย่าง
จุดเริ่มต้น	weight สุ่ม	โมเดลที่เทรนมาแล้ว
เครื่อง	GPU หลายพันตัว	GPU ตัวเดียว (Colab ฟรี)
เวลา	หลายสัปดาห์–เดือน	นาที–ชั่วโมง
ใครทำ	Meta, Alibaba, SCB 10X, ...	เรา (วันนี้ใน Lab 4)

ทั้งสองคือ gradient descent แบบสัปดาห์ 1 — ต่างที่ข้อมูลและจุดเริ่มต้น

- Fine-tuning เริ่มจากโมเดลที่เทรนมาแล้ว ใช้ข้อมูลหลักร้อยถึงหลักหมื่น ใช้ GPU ตัวเดียวเป็นนาทีถึงชั่วโมง — หลักการเดียวกับ transfer learning ที่ fine-tune ResNet กับดาต้าเซ็ตเล็กในสัปดาห์ 1

ขั้นที่ 3

LoRA: แช่แข็ง weight เดิม แล้วเทรนแค่ตารางผอม ๆ



- ปัญหาของ fine-tune เต็ม: ต้องเก็บ gradient + optimizer ของทุกค่า ≈ 16 ไบต์/พารามิเตอร์ (weight + gradient + ค่าที่ Adam ต้องจำ) ≈ 20 GB มากกว่า T4 (≈ 15 GB)

- LoRA เทรนเฉพาะ A, B แล้วรวมกลับเข้า W ตอนใช้ หรือเก็บแยกก็ได้

ขั้นที่ 3

ทำไม LoRA เทรนแค่ ~0.3% ก็พอ

- การปรับโมเดลให้ทำงานใหม่ ไม่ต้องขยับ weight ทุกทิศทาง — แค่ไม่กี่ทิศทางก็พอ (low-rank)
- แทนการเปลี่ยนแปลงของตาราง 2,048 × 2,048 (4.2 ล้านค่า) ด้วยตารางผอมสองตาราง 2,048 × r และ r × 2,048
  - r = 16 → 65,536 ค่า = 1.6% ของตารางเดิม
- ใส่ใน Q, K, V, O ครบ 16 ชั้น → รวม ≈ 3.4 ล้านค่า ≈ 0.28% ของโมเดล
  - K, V เล็กกว่า (2,048 × 16 + 16 × 512) เพราะกว้างแค่ 512

● ใน Lab 4 ให้นักศึกษาคำนวณตัวเลขนี้ก่อนรัน แล้วเทียบกับที่โค้ดพิมพ์ออกมา (คาดว่า ≈ 3.4 ล้าน)

ขั้นที่ 3

QLoRA และไฟล์ adapter

1 QLoRA = LoRA + โมเดลหลัก 4-bit

เก็บตัวเลขแต่ละตัวด้วย 4 บิตแทน 16 → ตาราง weight หลักเล็กลงราว 4 เท่า (embedding ยังเก็บ 16-bit) แลกกับความแม่นยำเล็กน้อย

ทำให้โมเดล 1B–8B fine-tune บน GPU T4 ของ Colab ฟรีได้

โมเดล 1B จริง ๆ ใช้ LoRA แบบ 16-bit ก็พอ — ในแล็บใช้ 4-bit เพื่อให้วิธีเดียวกันใช้กับโมเดลใหญ่กว่าได้

2 ผลลัพธ์คือไฟล์ adapter

LoRA ที่เทรนเสร็จเป็นไฟล์ไม่กี่สิบ MB แยกจากโมเดลหลัก

สลับ "ทักษะ" ได้โดยไม่ต้องเก็บโมเดลใหม่ทั้งตัว — โมเดลหลัก 1 ตัว + adapter หลายตัว

เวลาใช้งานจะรวม (merge) กลับเข้า weight เดิม หรือเก็บแยกแล้วบวกตอนรันก็ได้

ขั้นที่ 4

chat template คือ "บทละคร" ที่มีป้ายบอกว่าใครพูด

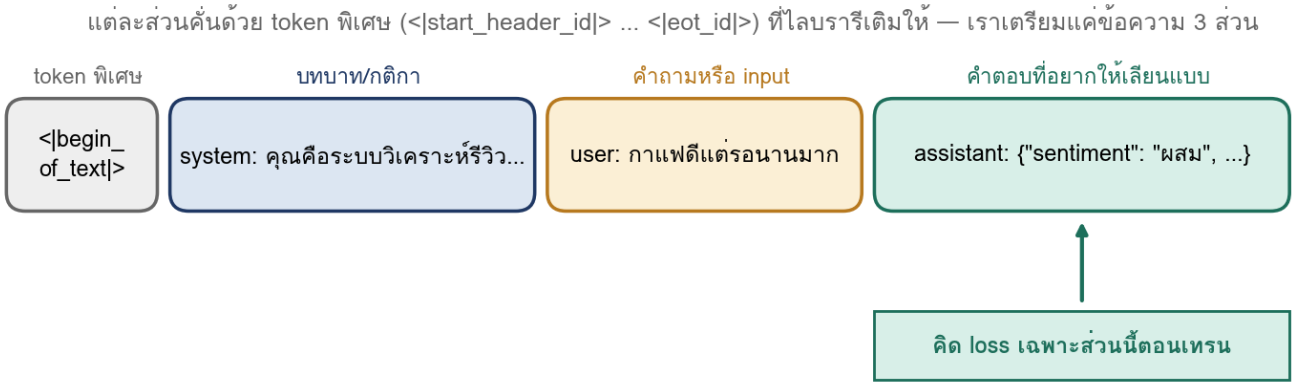
```
# สิ่งที่เราเขียน (3 ข้อความ)
[ผู้กำกับ / system] คุณคือระบบวิเคราะห์รีวิว
[ลูกค้า   / user]   กาแฟดีแต่รอนานมาก
[ผู้ช่วย  / assistant] {"sentiment": "ผสม", ...}

# สิ่งที่โมเดลเห็นจริง (ไลบรารีใส่ป้ายให้)
<|start_header_id|>system<|end_header_id|>
คุณคือระบบวิเคราะห์รีวิว<|eot_id|>
<|start_header_id|>user<|end_header_id|>
กาแฟดีแต่รอนานมาก<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
{"sentiment": "ผสม", ...}<|eot_id|>
```

- โมเดลรับได้แค่ข้อความยาวต่อกันแล้วเขียนต่อ — ไม่มีช่อง "คำสั่ง" แยก จึงใช้ป้ายพิเศษคั่นเหมือนชื่อตัวละครในบทละคร
- แต่ละป้ายเป็น token พิเศษ 1 token · <|eot_id|> = "จบบทพูดคนนี้" โมเดลพ่นออกมาเองเมื่อตอบเสร็จ
- โมเดลไม่ได้ "เข้าใจ" ว่าอะไรคือคำสั่ง — มันเห็นป้าย system/user/assistant มาเยอะจนรู้ว่าหลังป้ายไหนควรเขียนอะไร
- หลังป้าย assistant คือจุดที่โมเดลเริ่มเขียน — Lab 2 เห็นของจริงพร้อมบรรทัดวันที่ที่ template เติมเอง

ขั้นที่ 4

รูปแบบข้อมูลที่โมเดล instruct ต้องการ: chat template



- โมเดล instruct ถูกเทรนมาให้เห็นบทสนทนาในรูปแบบตายตัว
- เราเตรียมแค่ข้อความ 3 ส่วน (system / user / assistant) ไลบรารีเติม token พิเศษให้
- คิด loss เฉพาะส่วนคำตอบ

ขั้นที่ 4

chat template ของ Llama 3 / Typhoon2 หน้าตาจริง

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

Cutting Knowledge Date: December 2023
Today Date: 26 Jul 2024

คุณคือระบบวิเคราะห์รีวิวร้านกาแฟ<|eot_id|><|start_header_id|>user<|end_header_id|>

กาแฟดีแต่รอนานมาก<|eot_id|><|start_header_id|>assistant<|end_header_id|>

{"sentiment": "ผสม", "issues": ["กาแฟดี", "รอนานมาก"]}<|eot_id|>
```

- template จริงเติมบรรทัดวันที่ให้อัตโนมัติ (Today Date จะเป็นวันที่รันจริง — นักศึกษาจะเห็นเองใน Lab 2) · สองข้อที่ต้องตั้งให้ถูก: คิด loss เฉพาะส่วน assistant และตอนใช้งานต้องใช้ system prompt เดียวกับตอนเทรน

ขั้นที่ 4

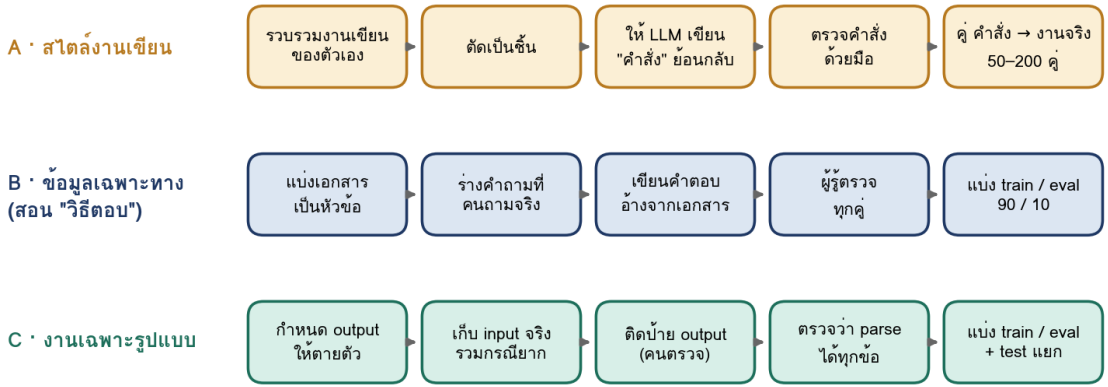
input กับ label ของ LLM มาจากไหน — ไม่ต้องมีใครติดป้าย

	โมเดลภาพ (สัปดาห์ 1)	โมเดลภาษา
input	รูปเสื้อ	คำทั้งหมดก่อนหน้า: "กาแฟ ดี แต่ รอ"
label	"เสื้อยืด" — คนต้องติดป้ายแยกทีละรูป	คำถัดไปในประโยคเดิม: "นาน" — ซ่อนอยู่ในข้อความอยู่แล้ว
1 ตัวอย่างให้	1 คู่ input → label	หลายคู่: กาแฟ→ดี · กาแฟ ดี→แต่ · กาแฟ ดี แต่→รอ · …
ตอน fine-tune แบบ chat	—	บทสนทนา 1 ก้อน = input · label = ก้อนเดียวกันเลื่อนไป 1 token โดย "ปิดตา" (mask) ส่วน system/user → คิด loss เฉพาะคำตอบ assistant

หลักการเดียวกับสัปดาห์ 1 ทุกประการ: มี input มี label เทียบกันแล้วปรับ weight — ต่างแค่ label มาจากข้อความเอง

ขั้นที่ 5

การเตรียมข้อมูล 3 กรณี — เป้าหมายเดียวกัน



เป้าหมายเดียวกันทุกกรณี: ได้คู่ input → output ที่โมเดลควรเลียนแบบ · output ต้องเป็นของจริง/คนตรวจแล้วเสมอ

- ได้ "คู่ input → output" ที่โมเดลควรเลียนแบบ
- output ต้องเป็นของจริงหรือคนตรวจแล้วเสมอ
- หลักร้อยตัวอย่างก็เริ่มเห็นผลสำหรับงานสไตล์และรูปแบบ

ขั้นที่ 5

JSON คืออะไร — และทำไมแล็บถึงให้โมเดลตอบเป็น JSON

```
{"sentiment": "ผสม",
 "issues": ["กาแฟดี", "รอนานมาก"],
 "price": 350}

# กฎมีแค่นี้
#   { }    = 1 ก้อนข้อมูล ข้างในเป็นคู่ "ชื่อ": ค่า
#   [ ]    = รายการหลายค่า
#   ค่า    = "ข้อความ" / ตัวเลข / true / false / ก้อนซ้อน

# โปรแกรมอ่านได้บรรทัดเดียว
obj = json.loads(output)
obj["sentiment"]         # → "ผสม"
obj["issues"][1]         # → "รอนานมาก"
```

- คำตอบของโมเดลมักถูกเอาไปใช้ต่อโดยโปรแกรม ไม่ใช่โดยคน — ถ้าตอบเป็นย่อหน้า โปรแกรมต้องเดาว่าตรงไหนคืออะไร
- ผิดรูปแบบนิดเดียว (ลืมปีกกา, มีข้อความห่อ) json.loads จะล้ม → ที่มาของตัวชี้วัด "parse ผ่าน" ใน Lab 2–3
- config.json ที่อ่านเมื่อเช้าก็คือไฟล์ JSON ไฟล์หนึ่ง
- token แรกของคำตอบ JSON เกือบต้องเป็น { เสมอ → Lab 2 จึงวัด P(ขึ้นต้น JSON) ได้ที่ token แรก

ขั้นที่ 5

ข้อควรระวังของแต่ละกรณี

กรณี	ตัวอย่างโจทย์	ข้อควรระวัง
A · สไตล์งานเขียน	อยากให้โมเดลเขียนบทความแบบสำนวนของเรา	อย่าให้ LLM เขียน output ให้ เพราะจะได้สไตล์ของ LLM ไม่ใช่ของเรา · ให้ LLM ช่วยได้เฉพาะ "เขียนคำสั่งย้อนกลับ" จากงานจริง
B · ข้อมูลเฉพาะทาง	คู่มือ/ระเบียบองค์กร อยากให้ตอบในแบบขององค์กร	fine-tune สอน "วิธีตอบ" (อ้างเลขข้อ ใช้ศัพท์ภายใน ปฏิเสธเมื่อไม่แน่ใจ) — ตัวข้อเท็จจริงควรมาจาก RAG เพราะ fine-tune ไม่ได้ "จำ" ได้แม่นและแก้ทีหลังยาก
C · งานเฉพาะรูปแบบ	สกัดข้อมูลเป็น JSON / จัดหมวดหมู่ (ต่อจากการบ้านรีวิวสัปดาห์ 2)	output ทุกข้อต้อง parse ได้ · คลาสไหนตัวอย่างน้อย โมเดลจะเรียนได้ไม่ดี · การกระจายของข้อมูลต้องใกล้กับที่จะใช้จริง

ขั้นที่ 6

ข้อผิดพลาดที่พบบ่อยตอน fine-tune

1 ข้อมูลน้อยแต่เทรนนาน

โมเดลท่องจำตัวอย่าง แล้วตอบอย่างอื่นแย่ลง = overfitting จากสัปดาห์ 1 กลับมาในร่างใหม่

แก้: มี eval set แล้วหยุดเมื่อ eval loss ไม่ลด

2 ข้อมูลไม่สะอาด

output สะกดผิด รูปแบบไม่ตรงกัน ตัวอย่างขัดแย้งกันเอง → โมเดลเรียนความไม่สม่ำเสมอไปด้วย

คุณภาพสำคัญกว่าจำนวน

3 ไม่มีชุดทดสอบก่อนเริ่ม

ไม่รู้ว่าดีขึ้นจริงหรือแค่รู้สึกว่าดีขึ้น

ควรมีชุดทดสอบ 10–20 ข้อพร้อมคำตอบที่ต้องการก่อนเทรนเสมอ — และแยกชุด "ปรับ prompt" (dev) กับชุด "ตัดสิน" (test)

4 ลืมเทียบกับ prompt ที่ดี

บางครั้ง prompt เต็ม (รูปแบบ + few-shot) ให้ผลเท่า fine-tune โดยไม่ต้องเทรนเลย

Lab 4 จะเทียบกันตรง ๆ บนชุด test เดียวกัน

สรุปส่วนที่ 2

หลังจบส่วนที่ 2 นักศึกษาควรพูดได้ว่า

"งานนี้ควร prompt / RAG / fine-tune เพราะ...
ถ้า fine-tune ผมต้องเตรียมคู่ input → output ในรูป chat template
ประมาณ ... ตัวอย่าง และวัดผลด้วยชุดทดสอบที่แยกไว้"

ลองเติมประโยคนี้ด้วยงานของตัวเอง 1 งาน

ส่วนที่ 3

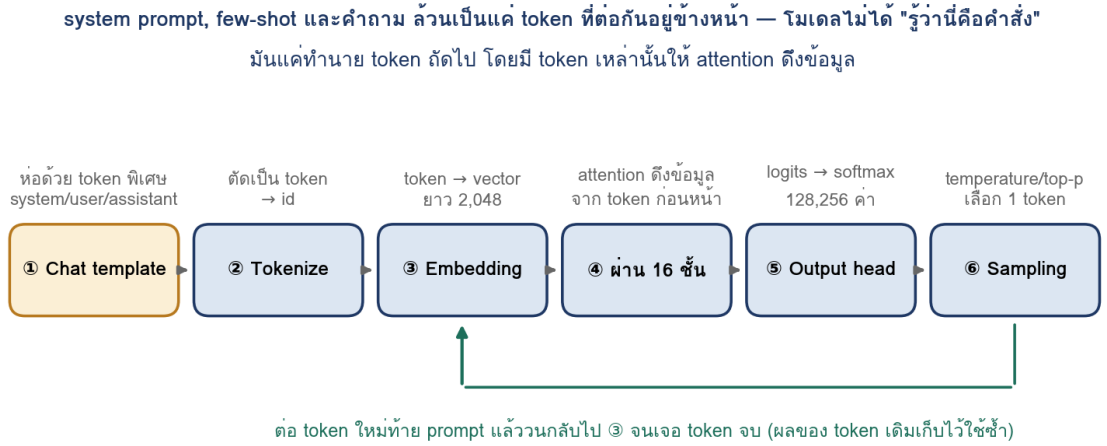
Prompt ทำงานอย่างไรในโมเดล

เปลี่ยนการ prompt จาก "เคล็ดลับที่ต้องท่อง" เป็นสิ่งที่อธิบายได้จากกลไก · prompt ดีกับไม่ดีต่างกันตรงไหน "ข้างใน" โมเดล

3

ขั้นที่ 1

เส้นทางของ prompt: 6 ขั้นจากข้อความถึงคำตอบ



- system prompt, few-shot และคำถาม ล้วนเป็นแค่ token ที่ต่อกันข้างหน้า — โมเดลไม่ได้ "รู้ว่านี่คือคำสั่ง" มันแค่ทำนาย token ถัดไป โดยมี token เหล่านั้นให้ attention ดึงข้อมูล

ขั้นที่ 1

ขั้น ①–③: ก่อนถึงโมเดล ทุกอย่างกลายเป็นตัวเลขแล้ว

- **① Chat template**
  - ข้อความถูกห่อด้วย token พิเศษ system / user / assistant แบบเดียวกับข้อมูลที่ใช้เทรน (ส่วนที่ 2 ขั้นที่ 4)
- **② Tokenize**
  - ตัดเป็น token แล้วแปลงเป็น id — จำนวน token คือสิ่งที่เสียเงินและกิน context (prompt ยาว = แพงขึ้น)
- **③ Embedding**
  - ทุก token กลายเป็น vector ยาว 2,048 — ณ จุดนี้ "คำสั่ง" กับ "คำถาม" ไม่มีอะไรต่างกันในสายตาโมเดล
- **④–⑥ ต่อไป**
  - ผ่าน 16 ชั้น (attention ดึงบริบทจาก token ก่อนหน้า) → logits → softmax → sampling เลือก 1 token แล้ววนกลับ ③

● logits = คะแนนดิบของทุก token ก่อนเข้า softmax (เห็นแล้วใน Lab 1.5 และจะใช้อีกใน Lab 2)

ขั้นที่ 2

ทำไม prompt ถึงมีผล — อธิบายจากกลไก 3 อย่าง

1 Attention ดึงได้เฉพาะสิ่งใน context

สิ่งที่ไม่ได้เขียนไว้ โมเดลต้อง "เดา" จากที่เคยเห็นตอนเทรน — สาเหตุหลักอย่างหนึ่งของ hallucination (สัปดาห์ 2)

→ การให้บริบทคือการเปลี่ยนจาก "เดา" เป็น "ดึง"

2 Instruction tuning ทำให้ตอบสนองรูปแบบคำสั่ง

post-training (base → instruct, RLHF ในสัปดาห์ 2) ให้โมเดลเห็นตัวอย่างจำนวนมากว่า "คำสั่งแบบนี้ตามด้วยคำตอบแบบนี้"

→ คำสั่งชัด + ระบุรูปแบบ = "เรียก" รูปแบบที่มันเคยเรียน

3 Sampling เลือกจากการกระจายความน่าจะเป็น

prompt คลุมเครือ → ความน่าจะเป็นกระจายหลายทาง → ผลเปลี่ยนทุกครั้งที่รัน

prompt ชัด → เกาะกลุ่ม → ผลนิ่ง (temperature/top-p จาก Lab 3 สัปดาห์ 2 คุมความเด็ดขาดในการเลือก)

ขั้นที่ 3

prompt ดี vs ไม่ดี: โจทย์เดียวกัน (สรุปรีวิวเป็น JSON)

```
# A · คลุมเครือ (ไม่ได้บอกด้วยซ้ำว่าอยากได้ JSON)
สรุปรีวิวนี้หน่อย: กาแฟดีแต่รอนานมาก

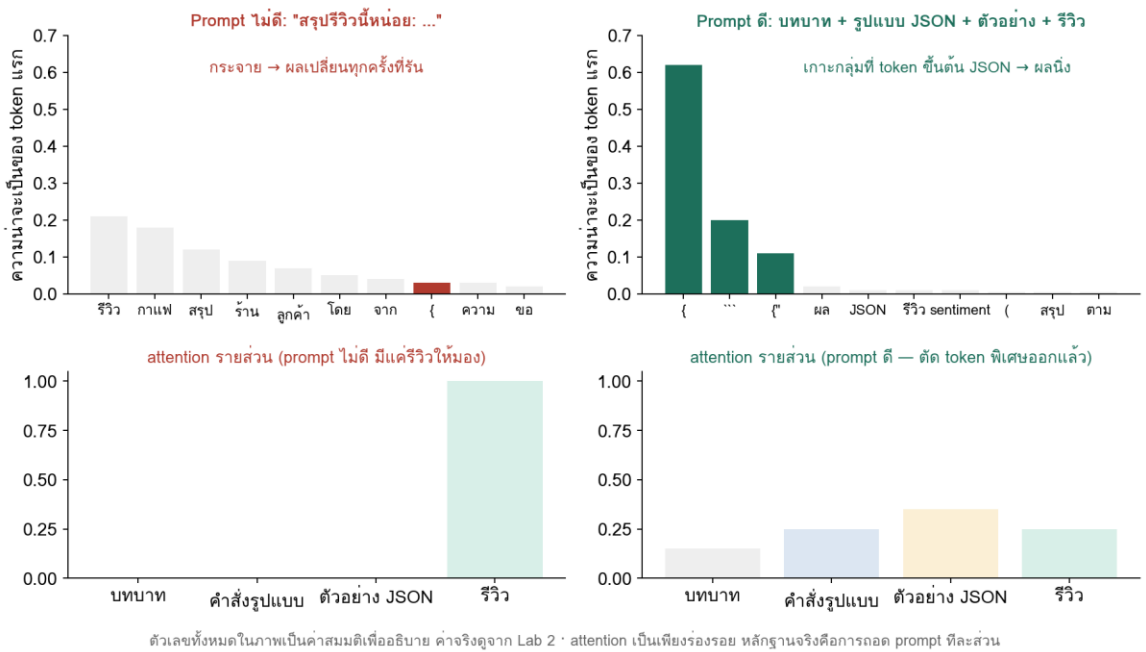
# B · สั้น (ขอ JSON แต่ไม่บอกรูปแบบ ไม่มีตัวอย่าง)
สรุปรีวิวนี้เป็น JSON: กาแฟดีแต่รอนานมาก

# C · เต็ม (4 ส่วน)
[บทบาท]    คุณคือระบบวิเคราะห์รีวิวร้านกาแฟ
[รูปแบบ]   ตอบเป็น JSON เท่านั้น ไม่ต้องอธิบาย
           field: sentiment (บวก/ลบ/ผสม), issues
[ตัวอย่าง] รีวิว "ขนมอร่อย ที่จอดรถหายาก" →
           {"sentiment": "ผสม",
            "issues": ["ขนมอร่อย", "ที่จอดรถหายาก"]}
[รีวิว]    ตอนนี้วิเคราะห์รีวิวนี้: กาแฟดีแต่รอนานมาก
```

- ระดับ B อยู่ตรงกลางเพื่อให้การเทียบยุติธรรม — ถ้า B ได้ผลใกล้ C แปลว่าโจทย์นี้แค่ "ขอ JSON" ก็พอ
- เก็บ C แยกเป็น 4 ส่วน เพราะ Lab 2 จะ "ถอด" ทีละส่วนเพื่อดูว่าส่วนไหนทำงาน
- C ยาวกว่า A หลายเท่า = ต้นทุน token ที่แลกมา
- ต่อไปจะตามทั้งสามระดับไปตามเส้นทาง 6 ขั้น (ตัวอย่างในสไลด์ถัดไปเทียบแค่ A กับ C ให้เห็นภาพ)

ขั้นที่ 3

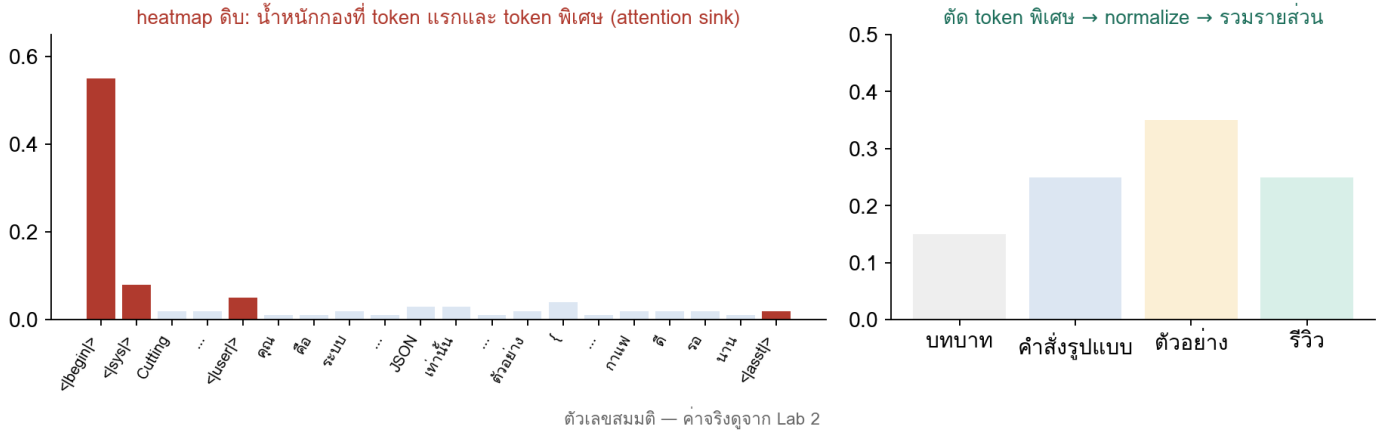
ตามดูทีละขั้น: ที่ token แรกของคำตอบต่างกันอย่างไร



- ขั้น ④: attention รายส่วนไปตกที่ตัวอย่างและคำสั่งรูปแบบ
- ขั้น ⑤: prompt ไม่ดี → ความน่าจะเป็นกระจาย, prompt ดี → เกาะกลุ่มที่ token ที่ขึ้นต้น JSON
- "กระจาย/เกาะกลุ่ม" วัดเป็นตัวเลขเดียวด้วย entropy (บิต): 0 = มั่นใจ token เดียว, 3 ≈ ลังเลระหว่าง 8 token (ตัวเลขในภาพสมมติ ค่าจริงดูใน Lab 2)

ขั้นที่ 3

ข้อควรระวังตอนดู attention: attention sink

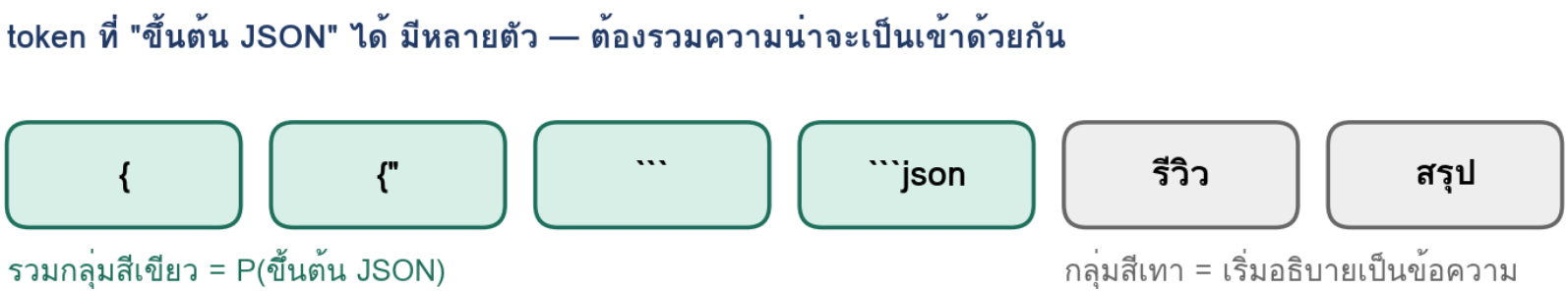


ตัวเลขสมมติ — ค่าจริงดูจาก Lab 2

- heatmap ดิบมักเห็นน้ำหนักกองที่ token แรกและ token พิเศษ — ปกติของโมเดลตระกูลนี้
- ต้องตัด token พิเศษออก normalize ใหม่ แล้วรวมเป็นรายส่วน
- attention เป็นเพียง "ร่องรอย" ไม่ใช่คำอธิบาย — หลักฐานจริงคือการถอด prompt ทีละส่วน

ขั้นที่ 3

ระวังตอนวัดความน่าจะเป็น: "ขึ้นต้น JSON" มีหลาย token



- tokenizer อาจมี {" เป็น token เดียว และโมเดลเล็กมักขึ้นต้นด้วย ```json — ดูแค่ "{" ตัวเดียวจะได้ค่าต่ำทั้งที่โมเดลกำลังจะตอบ JSON

- ตอน parse ให้ตัด ``` ออกก่อน json.loads

ขั้นที่ 3

ขั้น ⑥: รัน 5 ครั้ง แล้วถอดทีละส่วน

- **ทุกระดับ (A/B/C) วัดด้วยตัวชี้วัดชุดเดียวกัน**
  - จำนวน token (ต้นทุน) · entropy ของ token แรก (กระจาย/เกาะกลุ่ม) · P(ขึ้นต้น JSON) · อัตรา parse ผ่านเมื่อรัน 5 ครั้งที่ temperature 0.7
  - คาด: A → C token เพิ่ม entropy ลด P(JSON) เพิ่ม parse ผ่านเพิ่ม — แต่ค่าจริงอาจไม่สวย และนั่นคือประเด็นอภิปราย
- **ถอด prompt C ทีละส่วน (เหลือ 3 ส่วน)**
  - – บทบาท / – คำสั่งรูปแบบ / – ตัวอย่าง แล้ววัดด้วยตัวชี้วัดชุดเดิม
  - นี่คือหลักฐานที่หนักแน่นที่สุดว่า "ส่วนไหนของ prompt ทำงาน"
- **คำตอบไม่จำเป็นต้องเหมือนกันทุกโจทย์**
  - โจทย์ที่ต้องการรูปแบบ → คำสั่งรูปแบบและตัวอย่างมักสำคัญที่สุด · โจทย์ที่ต้องการน้ำเสียง → บทบาทสำคัญกว่า

● ทั้งหมดนี้คือ Lab 2 ของบ่ายนี้ — นักศึกษาจะได้ตัวเลขจริงของตัวเอง

ขั้นที่ 4

หลักการ prompt 5 ข้อ — แต่ละข้อมีเหตุผลจากกลไก

หลักการ	ทำอย่างไร	เหตุผลจากกลไก
ให้บริบท	ใส่ข้อมูลที่ต้องใช้ตอบลงไปใน prompt	attention ดึงได้เฉพาะสิ่งใน context · ลดการเดา = ลด hallucination
ระบุรูปแบบ output	บอกชัด: JSON / ตาราง / 3 ข้อ / ไม่เกิน 50 คำ	โมเดลทำนาย "รูปแบบ" เก่งกว่าทำนาย "ความตั้งใจ" ที่ไม่ได้พูด
ให้ตัวอย่าง (few-shot)	ใส่คู่ input → output 1–3 คู่ก่อนโจทย์จริง	เรียนรูปแบบจาก context ได้ทันที (in-context learning) เหมือน fine-tune ชั่วคราวโดยไม่เปลี่ยน weight
ให้คิดเป็นขั้น (CoT)	"อธิบายเหตุผลทีละขั้นก่อนสรุป"	โมเดลทำนายทีละ token — ให้มันเขียนขั้นกลางออกมา = token ถัดไปมีข้อมูลระหว่างทางให้ดึง (โมเดลเล็กได้ประโยชน์น้อยกว่า)
กำหนดบทบาท	"คุณคือนักบัญชีที่อธิบายให้คนทั่วไปฟัง"	เปลี่ยนการกระจายความน่าจะเป็นไปยังกลุ่มคำและน้ำเสียงที่พบคู่กับบทบาทนั้น

- Lab 3 ไล่ระดับทีละตัวแปร: L0 zero-shot → L1 + รูปแบบ → L2 + few-shot → L3 + บทบาท (CoT ทดลองแยก) แล้ววัดคะแนนบนชุด dev — คะแนนนับ "รูปแบบถูก" ด้วย

ขั้นที่ 5

กรอบสรุปของวัน

Fine-tune เปลี่ยน "ตัวโมเดล"
Prompt เปลี่ยน "คำสั่ง" ใน input
RAG เปลี่ยน "ข้อมูลอ้างอิง" ใน input (ค้นมาใส่ให้อัตโนมัติ)

ทั้งหมดคือการแก้หนึ่งในสองตำแหน่งของสมการ คำตอบ = โมเดล(input) · RAG เป็นเนื้อหาสัปดาห์ 4
บ่ายนี้ลงมือ prompt (Lab 2–3) และ fine-tune (Lab 4) แล้วปิดด้วยการเทียบ 3 แนวทาง:
(1) zero-shot (2) prompt เต็ม (รูปแบบ + few-shot) (3) fine-tune + prompt สั้น

สรุปส่วนที่ 3

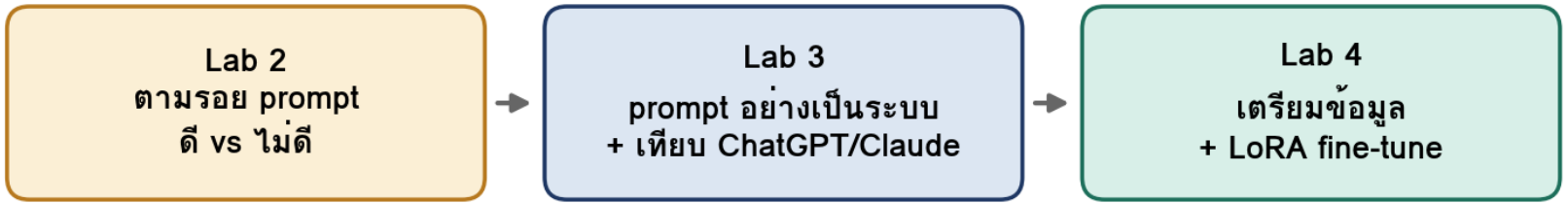
หลังจบส่วนที่ 3 นักศึกษาควรพูดได้ว่า

"prompt นี้ดีกว่าเพราะให้บริบทที่ attention ดึงได้ระบุรูปแบบที่โมเดลเคยเห็น และทำให้ความน่าจะเป็นของ token แรกเกาะกลุ่ม ผลจึงนิ่งและถูกรูปแบบ"

ไม่ใช่ "เพราะมันเป็นเคล็ดลับที่เขาบอกกันมา"

ช่วงบ่าย

บ่ายนี้ทำอะไร



ทุกแล็บใช้ Typhoon2-1B บน Colab T4 · Lab 1 (ผ่าโมเดล) ทำไปแล้วตอนเช้า

- ทุกแล็บบน Colab T4 ด้วย Typhoon2-1B
- Lab 3 ใช้ชุด dev 10 ข้อปรับ prompt และ Lab 4 เปิดชุด test 20 ข้อครั้งแรกเพื่อเทียบ 3 แนวทาง
- เซลล์ติดตั้งของ Lab 4 ให้รันในช่วงพักก่อนเริ่ม Lab 4 (ใช้ 2–5 นาที)

สรุป

สรุปช่วงเช้า 3 ข้อ

- **1 · LLM = ฟังก์ชันทำนาย token ถัดไป ประกอบจากชิ้นส่วน 4 ประเภทที่ล้วนเป็น neural network แบบสัปดาห์ 1**
  - Typhoon2-1B = โครงสร้าง Llama 3.2 (16 ชั้น กว้าง 2,048 ≈ 1.24B) ที่เทรนต่อด้วยข้อมูลไทย
- **2 · จะสอนโมเดลเพิ่ม มีสองที่ให้แก้: input (prompt, RAG) หรือ weight (fine-tune ด้วย LoRA ≈ 0.3% ของโมเดล)**
  - ข้อมูล fine-tune = คู่ input → output ในรูป chat template · แยกชุด dev / test เสมอ
- **3 · prompt มีผลเพราะ attention ดึงได้เฉพาะสิ่งใน context, instruction tuning ตอบสนองรูปแบบคำสั่ง และ sampling เลือกจากการกระจาย**
  - prompt ดี = ทำให้ความน่าจะเป็นของ token แรก "เกาะกลุ่ม" — วัดได้จริงใน Lab 2